

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

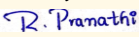
Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1002	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	RAVURU PRANATHI	Reg. No.:	192525076
Date:	21-08-2026	Duration:	60 minutes

Code of Conduct

I **R.Pranathi(19252506)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

1. INTRODUCTION

The **Smart Construction Project Management System (SCPMS)** is a software system designed to simplify and automate construction project activities. The system manages important information such as construction projects, tasks, workers, materials, budgets, schedules, progress, notifications, and reports. A construction project management system may be updated frequently to add new features, fix defects, improve security, and support changing project requirements. Manual building, testing, and deployment can introduce errors and consume considerable time.

Therefore, a **Continuous Integration and Continuous Deployment (CI/CD) pipeline** is proposed to automate the software delivery process. The pipeline automatically integrates source code, builds the application, performs testing, checks code quality and security, packages the application, deploys it to staging, and finally deploys the approved version to production.

The proposed workflow is:

Code Commit → Build → Unit Test → Integration/API Test → Code Quality → Security Scan → Package → Staging → Acceptance Test → Production → Monitoring → Rollback if Required

2. OBJECTIVES

The main objectives of the proposed CI/CD pipeline are:

1. To automate the integration of source code.
2. To automatically build the Smart Construction Project Management System.
3. To identify defects early through automated testing.
4. To maintain consistent code quality.
5. To identify security vulnerabilities before production deployment.
6. To reduce manual deployment effort.
7. To provide separate development, staging, and production environments.
8. To deploy only tested and approved software.
9. To provide monitoring and notifications after deployment.
10. To provide a reliable rollback mechanism when deployment fails.
11. To reduce deployment time and human errors.
12. To support frequent and reliable software releases.

3. PROJECT DESCRIPTION

The Smart Construction Project Management System provides a centralized platform for managing construction activities.

3.1 Major Modules

The system contains the following modules:

1. User Authentication
2. Role-Based Access Control
3. Project Management
4. Task Management
5. Worker Management
6. Material Management
7. Budget and Expense Management
8. Schedule Management
9. Progress Tracking
10. Notifications and Alerts
11. Dashboard
12. Report Generation
13. Search and Filtering

3.2 Main Users

User	Responsibilities
Administrator	Manage users, roles, and system settings
Project Manager	Manage projects, budgets, schedules, tasks, and reports
Site Engineer	Monitor construction activities and progress
Supervisor	Assign and monitor workers and tasks
Worker	View assigned tasks and update task status

4. REQUIREMENT ANALYSIS

The CI/CD pipeline requirements are derived from the characteristics of the Smart Construction Project Management System.

4.1 Application Requirements

Requirement	Description	CI/CD Need
Project Management	Create and update construction projects	Automated functional testing
Task Management	Assign and monitor tasks	Unit and integration testing
Worker Management	Maintain worker information	API and database testing
Material Management	Track material quantities	Validation and boundary testing
Budget Management	Calculate expenses and remaining budget	Automated calculation tests
Schedule Management	Maintain project deadlines	Date validation tests
Progress Tracking	Track project completion	Boundary value testing
Authentication	Secure user login	Security and authentication tests
Role Management	Restrict access by role	Automated authorization tests
Reports	Generate project reports	System and integration tests
Notifications	Alert users about deadlines and resources	Integration testing
Frequent Updates	Continuous feature development	Continuous Integration
Reliable Releases	Avoid defective deployments	Automated CI/CD
Security	Protect construction and user data	Security scanning
Recovery	Restore previous stable version	Rollback mechanism

5. CI/CD REQUIREMENTS

Based on the above analysis, the following CI/CD requirements are identified.

CR01 – Source Control

All source code must be maintained using a version-control system.

CR02 – Automated Integration

Every Pull Request or approved commit should automatically trigger the CI pipeline.

CR03 – Automated Build

The application must be automatically compiled and packaged.

CR04 – Automated Testing

Unit, integration, API, system, smoke, and regression tests should be executed automatically.

CR05 – Code Quality

Source code should pass automated code-quality analysis.

CR06 – Security

Dependencies, source code, and container images should be checked for security vulnerabilities.

CR07 – Packaging

The tested application should be packaged into a versioned Docker image.

CR08 – Staging Deployment

The application should first be deployed to a staging environment.

CR09 – Production Deployment

Only an approved and validated build should be deployed to production.

CR10 – Monitoring

The production application should be monitored continuously.

CR11 – Notification

Developers should be notified about build, testing, deployment, and failure results.

CR12 – Rollback

The system should be able to return to the previous stable version when a production failure occurs.

6. PROPOSED CI/CD TOOLS AND TECHNOLOGIES

Pipeline Stage	Tool/Technology	Purpose
Source Control	Git	Version control
Remote Repository	GitHub	Code storage and collaboration
CI/CD	GitHub Actions	Automating pipeline stages
Backend Build	Maven	Build Java backend if applicable
Frontend Build	npm	Build frontend application
Unit Testing	JUnit / Jest	Unit testing
API Testing	Postman/Newman	Automated API testing
Code Quality	SonarQube	Code-quality analysis
Dependency Security	OWASP Dependency-Check	Detect vulnerable dependencies
Containerization	Docker	Package application
Container Registry	GitHub Container Registry	Store Docker images
Deployment	Docker/Kubernetes	Application deployment
Monitoring	Prometheus	Collect system/application metrics
Visualization	Grafana	Monitoring dashboards
Notification	Email/Teams/Slack	Pipeline notifications

7. JUSTIFICATION OF TOOL SELECTION

7.1 Git

Git is selected because it provides version control, branching, merging, and history tracking. It allows developers to work on separate features without affecting the main application.

7.2 GitHub

GitHub provides:

- Remote source-code storage
- Pull Requests
- Code reviews
- Branch management
- Collaboration
- Integration with GitHub Actions

7.3 GitHub Actions

GitHub Actions is selected as the CI/CD automation tool because it integrates directly with GitHub.

It can automatically:

- Build the application
- Run tests
- Perform security checks
- Build Docker images
- Deploy applications
- Send notifications

7.4 SonarQube

SonarQube is used to detect:

- Bugs
- Code smells
- Security issues
- Duplicated code
- Maintainability problems

A quality gate prevents poor-quality code from continuing through the pipeline.

7.5 Docker

Docker packages the application and its dependencies into a container.

This provides consistency between:

Development → Staging → Production

7.6 OWASP Dependency-Check

It identifies known vulnerabilities in third-party dependencies used by the application.

7.7 JUnit/Jest

These tools are used for automated unit testing.

Examples include:

- Budget calculations
- Progress calculations
- Login validation
- Material quantity validation

7.8 Postman/Newman

Postman/Newman is used for automated API testing.

It can test APIs related to:

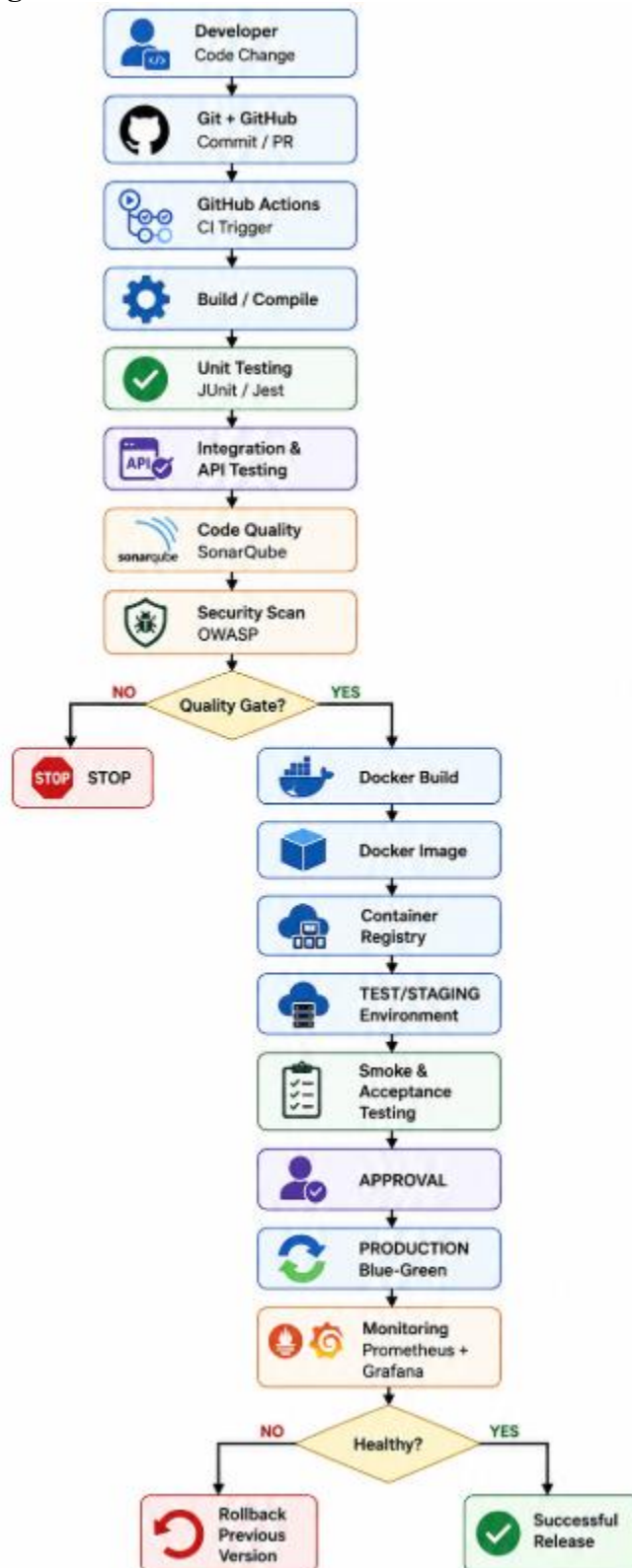
- Login
- Projects
- Tasks
- Workers
- Materials
- Budgets
- Reports

7.9 Prometheus and Grafana

Prometheus collects system and application metrics, while Grafana provides dashboards for monitoring system health.

8. CI/CD PIPELINE ARCHITECTURE

Main Pipeline Diagram

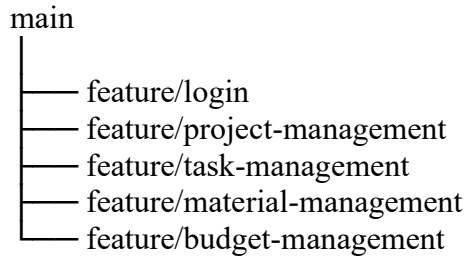


9. PIPELINE WORKFLOW

Stage 1: Source Code Management

Developers create feature branches from the main branch.

Example:



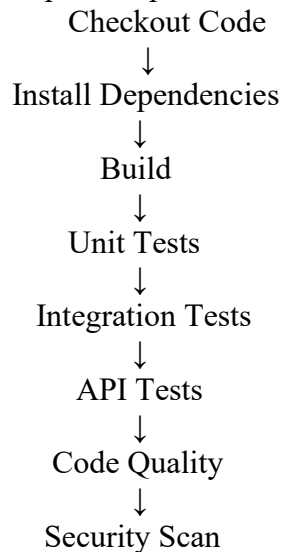
After completing a feature, the developer creates a Pull Request. Code review is performed before merging into the main branch.

10. STAGE 2: CONTINUOUS INTEGRATION

GitHub Actions automatically starts when:

- A Pull Request is created.
- A Pull Request is updated.
- Code is pushed to the main branch.

The CI process performs:



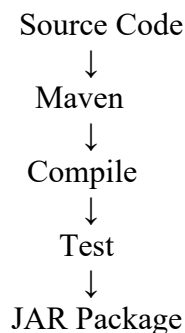
If a critical stage fails, the pipeline stops.

11. STAGE 3: BUILD

The application is automatically built.

Backend

If the backend uses Java:



Frontend

If the frontend uses JavaScript:

Source Code



npm install



npm test



npm run build



Production Build

The exact build command depends on the technology used to implement SCPMS.

12. STAGE 4: AUTOMATED TESTING

Automated testing is integrated directly into the CI pipeline.

12.1 Unit Testing

Individual functions are tested.

Example: Budget Calculation

Total Budget = ₹10,00,000

Total Expense = ₹2,00,000

Expected Remaining Budget = ₹8,00,000

The test automatically verifies the calculation.

Other Unit Tests

- Login validation
- Progress calculation
- Material quantity validation
- Expense calculation
- Date validation

13. INTEGRATION TESTING

Integration testing verifies communication between modules.

Examples:

Project Module



Task Module



Worker Module

and:

Material Module



Inventory Database

and:

Budget Module



Expense Database

The objective is to verify that interconnected modules work correctly.

14. API TESTING

API testing verifies communication between the frontend and backend.

Example APIs:

POST /login

POST /projects

GET /projects

PUT /projects/{id}

POST /tasks

PUT /tasks/{id}/status

POST /materials

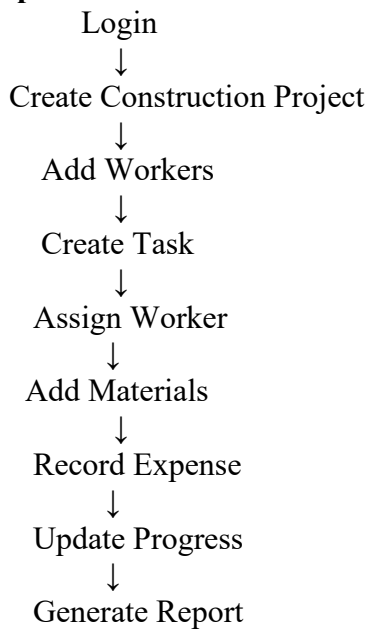
GET /reports

Postman/Newman can automatically execute these API tests in the CI pipeline.

15. SYSTEM TESTING

System testing verifies the complete application workflow.

Example



The entire workflow must complete successfully.

16. SMOKE TESTING

Smoke tests are executed after deployment to staging.

Important checks include:

1. Application starts successfully.
2. Login works.
3. Dashboard loads.
4. Database connection works.
5. Project creation works.
6. Task creation works.
7. API endpoints respond correctly.

If smoke testing fails, production deployment is blocked.

17. REGRESSION TESTING

Regression testing verifies that new changes do not break existing features.

Example

If a new **Material Management** feature is added, regression tests verify that:

- Project Management still works.
- Task Management still works.
- Budget Management still works.
- Worker Management still works.
- Reports still work.

18. CODE QUALITY ANALYSIS

SonarQube is executed after building and testing.

Source Code



SonarQube



Quality Analysis



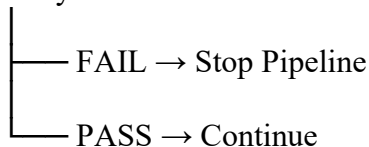
Quality Gate

The quality gate checks:

- Bugs
- Vulnerabilities
- Code smells
- Duplicated code
- Maintainability
- Test coverage

Decision

Quality Gate



19. SECURITY INTEGRATION

Security checks are performed before packaging and deployment.

Security Checks

1. Dependency Scanning

Checks third-party libraries for known vulnerabilities.

2. Static Security Analysis

Checks source code for potential security issues.

3. Secret Detection

Prevents passwords, API keys, and tokens from being committed.

4. Container Scanning

Docker images are scanned before deployment.

5. Access Control

Only authorized developers and administrators can modify or deploy production code.

20. SECRET MANAGEMENT

Sensitive information must not be stored directly in the source code.

Examples:

DATABASE_PASSWORD

JWT_SECRET

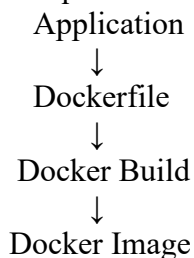
API_KEY

CLOUD_CREDENTIALS

These should be stored using secure GitHub repository/environment secrets.

21. STAGE 5: PACKAGING

After all required tests and quality checks pass, Docker packages the application.



Example version:

scpms:v1.0.0

scpms:v1.1.0

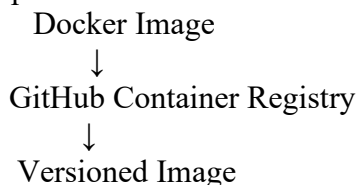
scpms:v1.2.0

Versioned images make deployment and rollback easier.

22. CONTAINER REGISTRY

The Docker image is stored in a container registry.

Example:



The same tested image is then promoted from staging to production.

This prevents different builds from being used in different environments.

23. DEPLOYMENT ENVIRONMENTS

Three major environments are used.

23.1 Development Environment

Used by developers.

Developer



Development

Purpose:

- Feature development
- Local testing
- Debugging

23.2 Staging/Test Environment

Used for final validation.

CI Pipeline



Staging



Smoke Testing



Acceptance Testing

The staging environment should closely resemble production.

23.3 Production Environment

Used by actual users.

Approved Image



Production



Real Users

Only a successfully tested and approved version reaches production.

24. DEPLOYMENT FLOW

Development



Pull Request



CI Build



Automated Testing



Quality + Security



Docker Package



Staging



Acceptance Testing



Approval



Production

25. DEPLOYMENT STRATEGY – BLUE-GREEN DEPLOYMENT

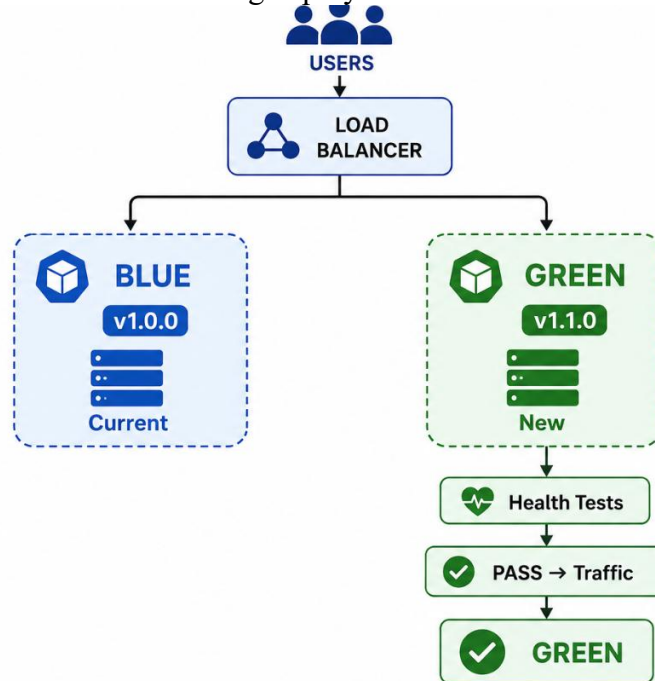
A Blue-Green Deployment strategy is proposed.

Blue

Blue represents the current stable production version.

Green

Green represents the new version being deployed.



Deployment Process

1. Blue continues serving users.
2. Green is deployed with the new version.
3. Green is tested.
4. Health checks are performed.
5. If Green passes, user traffic is moved to Green.
6. Blue remains available temporarily.
7. If Green fails, traffic is returned to Blue.

Advantage

Blue-Green deployment provides:

- Reduced downtime
- Safer production deployment
- Quick rollback
- Easy comparison between versions

26. DATABASE DEPLOYMENT STRATEGY

The system contains important construction data such as:

- Project information
- Worker information
- Material stock
- Budget
- Expenses
- Tasks
- Progress

Therefore, database changes must be handled carefully.

Database Migration Process

Database Change



Migration Script



Test in Staging



Backup Production Database



Apply Migration



Deploy Application

A database backup should be taken before major production changes.

27. ROLLBACK STRATEGY

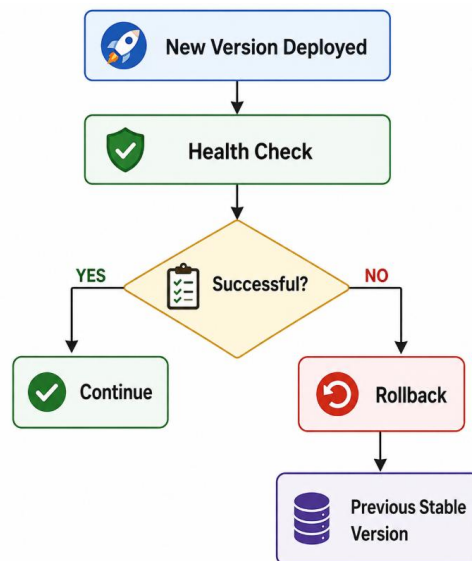
Rollback restores the previous stable version when the new version causes critical problems.

Rollback Conditions

Rollback should be initiated if:

- Application fails to start.
- Health check fails.
- Critical API fails.
- Database connection fails.
- Error rate becomes very high.
- Critical security vulnerability is discovered.
- Major functionality becomes unavailable.

Rollback Flow



Example:

Current Version = v1.1.0

Previous Version = v1.0.0

Failure in v1.1.0



Rollback



Restore v1.0.0

28. MONITORING

After production deployment, monitoring is performed continuously.

Metrics

- CPU usage
- Memory usage
- Response time
- Request rate
- Error rate
- Database health
- Application availability
- Container health

Monitoring Architecture

Production Application



Metrics



Prometheus



Grafana



Monitoring Dashboard

Monitoring allows the development team to identify problems quickly.

29. NOTIFICATION SYSTEM

The pipeline sends notifications when important events occur.

Notifications

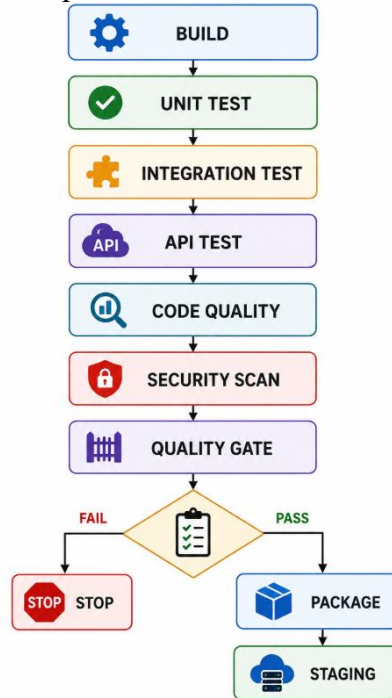
- Build successful
- Build failed
- Unit tests failed
- Security scan failed
- Quality gate failed
- Staging deployment completed
- Production deployment completed
- Rollback initiated

Possible channels:

- Email
- Microsoft Teams
- Slack

30. QUALITY GATES

The pipeline contains multiple checkpoints.



Production deployment is allowed only when required quality gates are passed.

31. COMPLETE PIPELINE STAGE TABLE

Stage	Activity	Tool	Output
1	Source Code	Git/GitHub	Versioned source
2	CI Trigger	GitHub Actions	Pipeline started
3	Build	Maven/npm	Build package
4	Unit Test	JUnit/Jest	Unit test results
5	Integration Test	JUnit/API tools	Integration results
6	API Test	Postman/Newman	API test results
7	Code Quality	SonarQube	Quality report
8	Security	OWASP tools	Security report
9	Package	Docker	Docker image
10	Registry	GitHub Container Registry	Versioned image
11	Staging	Docker/Kubernetes	Staging application
12	Smoke Test	Automated tests	Staging validation
13	Approval	Project/Release Manager	Deployment approval
14	Production	Docker/Kubernetes	Live application
15	Monitoring	Prometheus/Grafana	Health metrics
16	Notification	Email/Teams/Slack	Status notification
17	Rollback	Blue-Green	Previous version

32. PIPELINE FAILURE HANDLING

Failure	Action
Code compilation failure	Stop pipeline
Unit test failure	Stop pipeline
Integration test failure	Stop pipeline
API test failure	Stop pipeline
SonarQube quality gate failure	Stop pipeline
Critical vulnerability	Stop pipeline
Docker build failure	Stop pipeline
Staging smoke test failure	Block production deployment
Production health-check failure	Rollback
High production error rate	Investigate and rollback if required

33. SAMPLE CI/CD WORKFLOW

The following is a conceptual GitHub Actions workflow:

name: SCPMS CI/CD Pipeline

on:

push:

branches: [main]

pull_request:

branches: [main]

jobs:

build-test:

runs-on: ubuntu-latest

steps:

- name: Checkout Source Code
uses: actions/checkout@v4

- name: Install Dependencies
run: npm install

- name: Build Application
run: npm run build

- name: Run Unit Tests
run: npm test

- name: Security Check
run: npm audit

- name: Build Docker Image
run: docker build -t scpms:latest .

staging:

needs: build-test

runs-on: ubuntu-latest

steps:

- name: Deploy to Staging
run: echo "Deploying SCPMS to staging"
- name: Run Smoke Tests
run: echo "Running staging smoke tests"

production:

needs: staging
runs-on: ubuntu-latest

steps:

- name: Deploy to Production
run: echo "Deploying SCPMS to production"
- name: Health Check
run: echo "Checking production health"

Note: This is a conceptual pipeline configuration. Actual commands should be modified according to the application's selected programming language, framework, database, and deployment platform.

34. END-TO-END WORKFLOW EXPLANATION

Step 1 – Developer writes code

A developer implements a feature such as Material Management.

Step 2 – Developer commits code

The code is committed to a feature branch.

Step 3 – Pull Request

A Pull Request is created to merge the feature into main.

Step 4 – CI starts

GitHub Actions automatically starts the pipeline.

Step 5 – Build

The application is compiled and built.

Step 6 – Automated Tests

Unit, integration, and API tests are executed.

Step 7 – Quality Check

SonarQube checks code quality.

Step 8 – Security Check

Dependencies and application components are scanned.

Step 9 – Package

Docker creates a versioned application image.

Step 10 – Staging

The image is deployed to the staging environment.

Step 11 – Acceptance Testing

Smoke and acceptance tests are executed.

Step 12 – Approval

The release is approved for production.

Step 13 – Production Deployment

Blue-Green deployment is used.

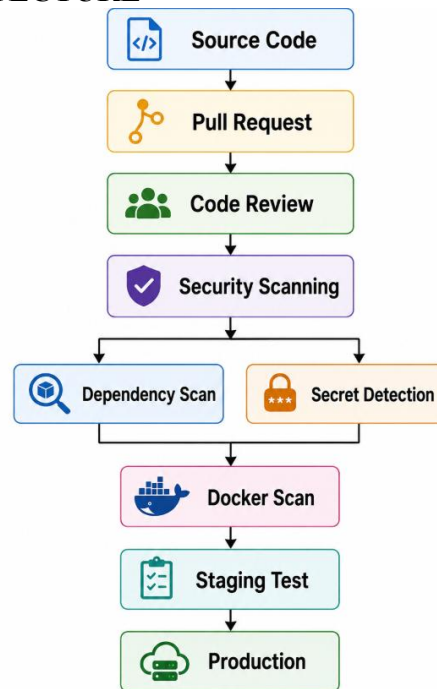
Step 14 – Monitoring

Prometheus and Grafana monitor the deployed application.

Step 15 – Rollback

If a serious problem is detected, traffic is returned to the previous stable version.

35. SECURITY ARCHITECTURE



Security is therefore treated as part of the pipeline rather than as a separate activity after deployment.

36. BENEFITS OF THE PROPOSED CI/CD PIPELINE

1. Faster software delivery – Releases can be automated.
2. Early defect detection – Bugs are identified immediately after code changes.
3. Improved code quality – Automated quality gates prevent poor-quality code.
4. Improved security – Vulnerability scanning is included in the pipeline.
5. Reduced manual effort – Build and deployment activities are automated.
6. Consistent deployment – Docker provides consistent environments.
7. Safer releases – Staging validation occurs before production.
8. Quick rollback – Blue-Green deployment supports rapid recovery.
9. Better collaboration – GitHub supports Pull Requests and code reviews.
10. Continuous monitoring – Production health is monitored after deployment.
11. Reliable construction data – Automated testing protects project, budget, material, and worker information.
12. Scalable architecture – Docker/Kubernetes can support future growth.

37. LIMITATIONS

1. Initial CI/CD setup requires time and technical knowledge.
2. Automated tests must be maintained as the application changes.
3. Cloud infrastructure may increase operational costs.
4. Incorrect pipeline configuration can cause deployment failures.
5. Database migrations require additional planning.
6. Monitoring and security tools require continuous maintenance.

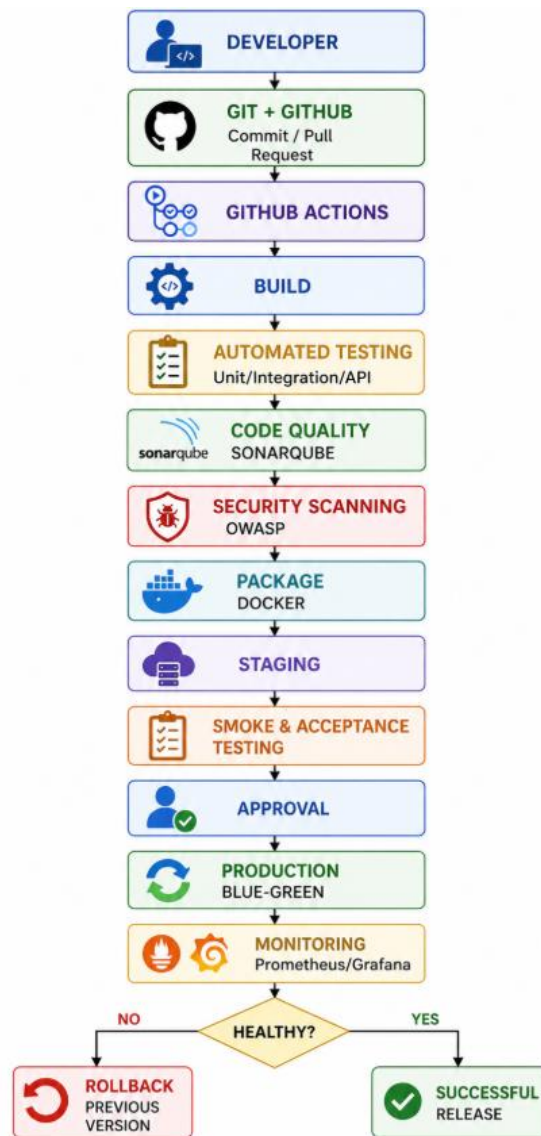
38. CI/CD PIPELINE SECURITY AND QUALITY CHECKPOINTS

Checkpoint	Purpose	Pipeline Action
Code Review	Verify code changes	Reject/approve PR
Build Check	Verify compilation	Stop if failed
Unit Tests	Verify individual functions	Stop if failed
Integration Tests	Verify modules	Stop if failed
API Tests	Verify backend APIs	Stop if failed
SonarQube	Verify code quality	Stop if quality gate fails
Security Scan	Detect vulnerabilities	Stop for critical issues
Docker Scan	Secure container	Stop for critical issues
Staging Test	Validate deployment	Block production if failed
Health Check	Verify production	Rollback if failed

39. REQUIREMENT-TO-PIPELINE TRACEABILITY

Project Requirement	CI/CD Implementation
Secure Login	Unit + integration + security tests
Project Management	Unit + API + system tests
Task Management	Unit + integration + API tests
Worker Management	API + database tests
Material Management	Unit + boundary + integration tests
Budget Management	Automated calculation tests
Schedule Management	Date validation tests
Progress Tracking	Boundary value tests
Notifications	Integration tests
Reports	System/API tests
Role-Based Access	Security and authorization tests
Frequent Feature Updates	Git + GitHub Actions
Reliable Releases	Automated testing + staging
Secure Application	Security scanning
Production Stability	Monitoring
Failed Release Recovery	Blue-Green rollback

40. OVERALL PIPELINE FLOW



CONCLUSION

The proposed **CI/CD Pipeline for the Smart Construction Project Management System** provides a complete automated software delivery process from source-code commit to production monitoring.

The pipeline integrates:

Git/GitHub → GitHub Actions → Build → Automated Testing → SonarQube → Security Scanning → Docker → Container Registry → Staging → Acceptance Testing → Production → Monitoring → Rollback

The pipeline is specifically designed for the construction project management domain. Automated tests verify important functions such as project creation, worker assignment, material tracking, budget calculations, scheduling, progress tracking, authentication, and report generation.

Separate development, staging, and production environments reduce deployment risk. Security scanning, code-quality gates, secret management, monitoring, and notifications improve reliability and security. The Blue-Green deployment strategy provides a quick rollback mechanism if the newly deployed version causes problems.

Therefore, the proposed CI/CD pipeline provides a systematic, automated, secure, reliable, and scalable **approach** for continuously integrating, testing, and deploying the Smart Construction Project Management System.

It satisfies the major CI/CD requirements of the assigned problem statement and provides complete coverage of the required source management, build, testing, quality analysis, security, packaging, deployment, monitoring, notification, and rollback stages.